

Doc. no.:	P2827R0
Date:	2022-3-14
Audience:	LEWG, LWG
Reply-to:	Zhihao Yuan <zy@miator.net>

Floating-point overflow and underflow in `from_chars` (LWG 3081)

- Floating-point overflow and underflow in `from_chars` (LWG 3081)
 - Motivation
 - Background
 - Technical Decisions
 - Implementation
 - Wording
 - Feature test macro
 - References

Motivation

When parsing floating-point numbers, I want to distinguish a failure between “unable to store in `double` because the ideal value is too large” and “unable to store in `double` because the ideal value is too small.” `std::from_chars`, as currently specified, cannot give this information.

`from_chars` writes to an output parameter `value` and returns `from_chars_result`.

```
struct from_chars_result {  
    const char* ptr;  
    errc ec;  
};
```

To quote from `[charconv.from.chars]` (<https://eel.is/c++draft/charconv#from.chars>):

If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` of the return value is equal to `errc::result_out_of_range`.

In short, one cannot even implement the Python interpreter's behavior using `from_chars`.

```
>>> 3.14e-2000
0.0
>>> -1.1e360
-inf
```

The status quo also creates false expectations for learners. Because when parsing integers, `errc::result_out_of_range` implies overflow. Then, when parsing floating-point numbers, few people realize that their code can report “number is too big” when the number is too small.

Background

LWG 3081 (<https://cplusplus.github.io/LWG/issue3081>) points out that users may encounter a loss of functionality when migrating from `strtod` to `from_chars`. This is largely true. In the case of `double`, the C standard requires `strtod` to return plus or minus `HUGE_VAL` and `errno` to acquire the value of `ERANGE` if the ideal value is too large, and to return “a value whose magnitude is no greater than the smallest normalized positive number” if the ideal value is too small, while whether `ERANGE` is set is implementation-defined. But in reality, it is becoming a defacto-standard for a decent `strtod` implementation to return a `0.` or `-0.` and set `errno` to `ERANGE`. Here is a snippet that runs on FreeBSD: [EaW53G \(https://godbolt.miator.net/z/EaW53G\)](https://godbolt.miator.net/z/EaW53G).
(self-host)

The C standard enables the following code to detect whether the number to parse is too large portably:

```
errno = 0;
n = strtod(p, NULL);
if (errno == ERANGE && (n == HUGE_VAL || n == -HUGE_VAL))
    /* number is too big */
```

However, I once saw a student come up with the following code:

```
errno = 0;
n = strtod(p, NULL);
if (errno == ERANGE && n != 0)
    /* number is too big */
```

It's interesting and sufficiently portable. It took advantage of the fact that both `0.` and `-0.` compared equal to `0` and reduced the criteria to a single test. Note that, pedantically speaking, you cannot test both positive and negative `HUGE_VAL` with `isinf` because there is no

guarantee that `HUGE_VAL` is not finite. To summarize, if we want to designate special values to distinguish underflow from overflow, $\pm 0.$ provides an advantage indeed.

Technical Decisions

Solving the problem requires finding a way to channel more error information back to the users of `from_chars`. So, we have design alternatives.

A. Throw an exception

Not in `<charconv>`.^[1]

B. Set a global variable

No.

C. Return some value other than `errc::result_out_of_range`

This breaks backward compatibility. It is reasonable for existing code to look specifically for `errc::invalid_argument` and `errc::result_out_of_range`; introducing a new error condition can change the meaning of such code.

D. Assign value to special values

Seems to be the only feasible thing to do.

“What special values” is the question. Here is a table to sort out the choices:

Idea	Underflow	Overflow
Python	$\pm 0.$	$\pm \text{inf}$
Popular <code>strtod</code> impl.	$\pm 0.$	$\pm \text{HUGE_VAL}$
<code>strtod</code>	$\pm [0, \text{finite_min_v}<\text{T}>]$	$\pm \text{HUGE_VAL}$
LWG 3081 ^[2]	$\pm [0, \text{finite_min_v}<\text{T}>]$	$\pm \text{finite_max_v}<\text{T}>$
P2827	$\pm 0.$	$\pm 1.$

The observation is that no special value is perfect without error handling. For example, even with a subnormal number, the relative quantization error can be significant. Therefore, this paper proposes an option that is portable, specified, and boolean-testable simultaneously but obviously mandates error handling.

As a probably unimportant detail, assigning any special value can be observed in some backward-incompatible fashion. A strict reading of the standard can tell you that “`from_chars` never assign `value` to a non-finite value.” So errors could be handled like this:

```
auto v = quiet_NaN_v<double>;
std::from_chars(first, last, v);
if (not std::isfinite(v))
    /* ec != errc() */
```

But this is rather atypical. Regardless, let's bump a version year in the feature testing macros.

Implementation

None at the time of writing, but as easy as this patch:

- libstdc++: Adjust `fast_float`'s over/underflow behavior for conformance (<https://gcc.gnu.org/git/?p=gcc.git;a=commitdiff;h=40b0d4472a2591cf27f3a81aa3fba57dc4532648>)

As you can see, the underlying algorithm has full knowledge of underflow/overflow, but today a standard library implementation must mask this fact to be compliant.

Wording

The wording is relative to N4928 (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4928.pdf>).

Modify `[charconv.from.chars]` (<https://eel.is/c++draft/charconv.from.chars>) as indicated:

[...] Otherwise, the characters matching the pattern are interpreted as a representation of a value of the type of `value`. The member `ptr` of the return value points to the first character not matching the pattern, or has the value `last` if all characters match. If the parsed value is not in the range representable by the type of `value`, `value` is unmodified unless otherwise specified and the member `ec` of the return value is equal to `errc::result_out_of_range`. [...]

[Drafting note: The behavior is retained when parsing integers. –end note]

```
from_chars_result from_chars(const char* first, const char* last, floating-point-t
                           chars_format fmt = chars_format::general);
```



Preconditions: `fmt` has the value of one of the enumerators of `chars_format`.

Effects: The pattern is the expected form of the subject sequence in the "c" locale, as described for `strtod`, except that

[...]

Let the value of the string matching the pattern be V . If V is not in the range representable by *floating-point-type*, `value` is assigned to

- `0.` if $V \in (0, 1)$, or
- `-0.` if $V \in (-1, 0)$, or
- `1.` if $V \in (1, \infty)$, or
- `-1.` if $V \in (-\infty, -1)$.

~~In any case, Otherwise,~~ the resulting `value` is one of at most two floating-point values closest to ~~the value of the string matching the pattern~~ V .

Feature test macro

Update values in [version.syn] (<https://eel.is/c++draft/version.syn>), header `<version>` synopsis:

[Drafting note: `from_chars` has no individual feature testing macro. —end note]

```
#define __cpp_lib_to_chars          201611L 20XXXXL // also in <charconv>
```

References

1. P0067R5 *Elementary string conversions, revision 5*. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0067r5.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0067r5.html>) ↩
2. LWG 3081 *Floating point `from_chars` API does not distinguish between overflow and underflow*. <https://cplusplus.github.io/LWG/issue3081> (<https://cplusplus.github.io/LWG/issue3081>) ↩